

Laboratory Work 4

SQL Queries

1. Retrieve all airline names in uppercase.

```
SELECT UPPER(airline_name)
FROM airline;
```

The screenshot displays a PostgreSQL client interface with the following components:

- Object Explorer:** A tree view on the left showing the database structure. The 'airline' table is selected under the 'public' schema.
- Query Editor:** The central pane shows the SQL query:

```
--1
SELECT UPPER(airline_name)
from airline;
```
- Data Output:** The bottom pane shows the results of the query. It displays a table with one column, 'upper text', and 207 rows. The first six rows are visible: KAZAIR, AIREASY, FLYHIGH, FLYFLY, GLOBAL AIRWAYS, and RANDOM AIRLINE 10....
- Status Bar:** At the bottom, it indicates 'Total rows: 207', 'Query complete 00:00:00.054', and a success message: 'Successfully run. Total query runtime: 54 msec. 207 row'.

upper text
1 KAZAIR
2 AIREASY
3 FLYHIGH
4 FLYFLY
5 GLOBAL AIRWAYS
6 RANDOM AIRLINE 10...

2. Replace any occurrence of the word "Air" in airline names with "Aero".

```
SELECT REPLACE(airline_name, 'Air', 'Aero')
FROM airline;
```

The screenshot shows a PostgreSQL client interface with the following components:

- Object Explorer:** A tree view on the left showing the database structure. The 'airline' table is selected under 'Tables (11)'. The 'airline' table has columns: `airline_id` (integer), `airline_code` (character varying), `airline_name` (character varying), `airline_country` (character varying), `created_at` (timestamp without time zone), and `updated_at` (timestamp without time zone).
- Query Editor:** The main area for writing SQL queries. The query being executed is:

```
--1
SELECT UPPER(airline_name)
from airline;

--2
SELECT REPLACE (airline_name, 'Air', 'Aero')
FROM airline;

--3
```
- Data Output:** A table showing the results of the query. The table has a header row 'replace text' and 6 data rows:

	replace text
1	KazAero
2	AeroEasy
3	FlyHigh
4	FlyFly
5	Global Aeroways
6	Random Aeroline 10...
- Messages:** A green message box at the bottom right states: 'Successfully run. Total query runtime: 56 msec. 207 rows affected.'
- Status Bar:** At the bottom, it shows 'Total rows: 207', 'Query complete 00:00:00.056', and 'Ln 6, Col 1'.

3. Find all flight numbers that correspond to airline ID 1 and airline ID 2.

```
SELECT flight_id
FROM flights
WHERE airline_id IN (1, 2);
```

The screenshot shows a PostgreSQL IDE interface with the following components:

- Object Explorer (Left):** Displays a tree view of the database schema. The 'Columns (12)' for the 'flights' table are listed: flight_id integer, sch_departure_time timestamp without time zone, sch_arrival_time timestamp without time zone, departing_airport_id integer, arriving_airport_id integer, departing_gate text, arriving_gate character varying(50), airline_id integer, act_departure_time timestamp without time zone, act_arrival_time timestamp without time zone, created_at timestamp without time zone, and updated_at timestamp without time zone.
- Query Editor (Center):** Contains a SQL query with six numbered comments:


```
--1
SELECT UPPER(airline_name)
from airline;

--2
SELECT REPLACE (airline_name, 'Air', 'Aero')
FROM airline;

--3
SELECT flight_id FROM flights
WHERE airline_id IN (1, 2);

--4
SELECT * from airport
WHERE airport_name LIKE '%Reginal%'
AND airport_name LIKE '%Air%';

--5
SELECT first_name, last_name,
       TO_CHAR(date_of_birth, 'month DD, YYYY')
FROM passengers;

--6
SELECT flight_id
```
- Data Output (Bottom):** Shows the results of the third query. The first column is 'flight_id [PK] integer'. The results are:

flight_id [PK] integer
1
2
3
4
5
6
- Status Bar (Bottom):** Displays 'Total rows: 80', 'Query complete 00:00:00.068', and a green message box: 'Successfully run. Total query runtime: 68 msec. 80 rows affected.'

4. Retrieve airports that contain the word "Reginal" and "Air" in their names.

```
SELECT *  
FROM airport  
WHERE airport_name LIKE '%Reginal%'  
      AND airport_name LIKE '%Air%';
```

The screenshot displays the pgAdmin 4 web interface. On the left, the 'Object Explorer' shows a tree view of the database schema, including tables like 'airport' and 'booking_flight'. The main pane is titled 'Query' and contains a SQL query. The query is as follows:

```
--1  
SELECT UPPER(airline_name)  
from airline;  
  
--2  
SELECT REPLACE (airline_name, 'Air', 'Aero')  
FROM airline;  
  
--3  
SELECT column_name, data_type  
FROM information_schema.columns  
WHERE table_name = 'flights'  
ORDER BY ordinal_position;  
  
SELECT column_name, data_type  
FROM information_schema.columns  
WHERE table_name = 'booking_flight'  
ORDER BY ordinal_position;  
  
SELECT * FROM flights;  
  
--4  
SELECT * from airport  
WHERE airport_name LIKE '%Reginal%'  
AND airport_name LIKE '%Air%';
```

Below the query editor, the 'Data Output' tab is active, showing the results of the query. The results are displayed in a table with the following columns: airport_id, airport_name, country, state, city, created_at, and updated_at. The table is currently empty, with 'Total rows: 0' indicated at the bottom. A status message at the bottom right states: 'Successfully run. Total query runtime: 54 msec. 0 rows affected.'

5. Retrieve passenger names and format birth dates as 'Month DD, YYYY'.

```
SELECT first_name, last_name,  
       TO_CHAR(date_of_birth, 'Month DD, YYYY')  
FROM passengers;
```

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays a tree view of the database schema, including tables like 'airline', 'baggage', 'booking', and 'passengers'. The 'Columns (12)' for the 'passengers' table are listed, including 'flight_id', 'sch_departure_time', 'sch_arrival_time', 'departing_airport_id', 'arriving_airport_id', 'departing_gate', 'arriving_gate', 'airline_id', 'act_departure_time', 'act_arrival_time', 'created_at', and 'updated_at'.

The main query editor displays the following SQL query:

```
11 FROM information_schema.columns  
12 WHERE table_name = 'flights'  
13 ORDER BY ordinal_position;  
14  
15 SELECT column_name, data_type  
16 FROM information_schema.columns  
17 WHERE table_name = 'booking_flight'  
18 ORDER BY ordinal_position;  
19  
20 SELECT * FROM flights;  
21  
22 --4  
23 SELECT * from airport  
24 WHERE airport_name LIKE '%Reginald%'  
25 AND airport_name LIKE '%Air%';  
26  
27 --5  
28 SELECT first_name, last_name,  
29        TO_CHAR(date_of_birth, 'month DD, YYYY')  
30 FROM passengers;  
31  
32 --6  
33 SELECT flight_id  
34 FROM flights  
35 WHERE act_arrival_time > sch_arrival_time;  
36
```

The 'Data Output' pane at the bottom shows the results of the query. It displays a table with three columns: 'first_name', 'last_name', and 'to_char'. The results are as follows:

	first_name	last_name	to_char
1	Test	Passenger	january 01, 2000
2	First1001	Last1001	july 03, 2005
3	First1002	Last1002	october 11, 1977
4	First1003	Last1003	march 22, 1980
5	First1004	Last1004	june 17, 1993
6	First1005	Last1005	november 14, 19...

The status bar at the bottom indicates 'Total rows: 201' and 'Query complete 00:00:00.061'. A green message box states: 'Successfully run. Total query runtime: 61 msec. 201 rows affected.'

6. Find all flight numbers that have been delayed based on actual arrival time.

```
SELECT flight_id
FROM flights
WHERE act_arrival_time > sch_arrival_time;
```

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays a tree view of the database schema, including tables like 'airline', 'baggage', 'boarding_pass', 'booking', 'booking_flight', and 'flights'. The 'flights' table is selected, and its columns are listed: flight_id (integer), sch_departure_time (timestamp with time zone), sch_arrival_time (timestamp without time zone), departing_airport_id (integer), arriving_airport_id (integer), departing_gate (text), arriving_gate (character varying(50)), airline_id (integer), act_departure_time (timestamp without time zone), act_arrival_time (timestamp without time zone), created_at (timestamp without time zone), and updated_at (timestamp without time zone).

The main query editor displays the following SQL query:

```
11 FROM information_schema.columns
12 WHERE table_name = 'flights'
13 ORDER BY ordinal_position;
14
15 SELECT column_name, data_type
16 FROM information_schema.columns
17 WHERE table_name = 'booking_flight'
18 ORDER BY ordinal_position;
19
20 SELECT * FROM flights;
21
22 --4
23 SELECT * from airport
24 WHERE airport_name LIKE '%Reginal%'
25 AND airport_name LIKE '%Air%';
26
27 --5
28 SELECT first_name, last_name,
29        TO_CHAR(date_of_birth, 'month DD, YYYY')
30 FROM passengers;
31
32 --6
33 SELECT flight_id
34 FROM flights
35 WHERE act_arrival_time > sch_arrival_time;
36
```

The 'Data Output' pane at the bottom shows the results of the query, displaying a single column 'flight_id' with a primary key constraint. A status message at the bottom right indicates: 'Successfully run. Total query runtime: 61 msec. 0 rows affected.'

7. Find flights that arrived late according to actual arrival time compared to scheduled arrival time.

```
SELECT flight_id, sch_arrival_time, act_arrival_time
FROM flights
WHERE act_arrival_time > sch_arrival_time;
```

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays a tree view of the database schema, including tables like 'airline', 'airport', 'baggage', 'boarding_pass', 'booking', 'booking_flight', 'flights', 'passengers', 'security_check', and 'students'. The 'flights' table is selected, and its columns are listed: flight_id (integer), sch_departure_time (timestamp with time zone), sch_arrival_time (timestamp without time zone), departing_airport_id (integer), arriving_airport_id (integer), departing_gate (text), arriving_gate (character varying(50)), airline_id (integer), act_departure_time (timestamp with time zone), act_arrival_time (timestamp without time zone), created_at (timestamp without time zone), and updated_at (timestamp without time zone).

The main query editor displays the following SQL query:

```
15 SELECT column_name, data_type
16 FROM information_schema.columns
17 WHERE table_name = 'booking_flight'
18 ORDER BY ordinal_position;
19
20 SELECT * FROM flights;
21
22 --4
23 SELECT * from airport
24 WHERE airport_name LIKE '%Reginal%'
25 AND airport_name LIKE '%Air%';
26
27 --5
28 SELECT first_name, last_name,
29 TO_CHAR(date_of_birth, 'month DD, YYYY')
30 FROM passengers;
31
32 --6
33 SELECT flight_id
34 FROM flights
35 WHERE act_arrival_time > sch_arrival_time;
36
37 --7
38 SELECT flight_id, sch_arrival_time, act_arrival_time
39 FROM flights
40 WHERE act_arrival_time > sch_arrival_time;
```

The 'Data Output' pane at the bottom shows the results of the query, with columns: flight_id [PK] integer, sch_arrival_time timestamp without time zone, and act_arrival_time timestamp without time zone. The status bar at the bottom indicates: 'Total rows: 0', 'Query complete 00:00:00.071', and a green message box stating 'Successfully run. Total query runtime: 71 msec. 0 rows affected.'

8. Show airlines from France, Portugal, or Poland created between 2023-11-01 and 2024-03-31.

```
SELECT *  
FROM airline  
WHERE airline_country IN ('France', 'Portugal', 'Poland')  
AND created_at BETWEEN '2023-11-01' AND '2024-03-31';
```

The screenshot shows a PostgreSQL IDE interface with the following components:

- Object Explorer:** A tree view on the left showing the database schema. The 'flights' table is selected, and its columns are listed: flight_id, sch_departure_time, sch_arrival_time, departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id, act_departure_time, act_arrival_time, created_at, and updated_at.
- Query Editor:** The main area for writing SQL queries. The query is:

```
SELECT * FROM flights;  
  
--4  
SELECT * from airport  
WHERE airport_name LIKE '%Reginal%'  
AND airport_name LIKE '%Air%';  
  
--5  
SELECT first_name, last_name,  
       TO_CHAR(date_of_birth, 'month DD, YYYY')  
FROM passengers;  
  
--6  
SELECT flight_id  
FROM flights  
WHERE act_arrival_time > sch_arrival_time;  
  
--7  
SELECT flight_id, sch_arrival_time, act_arrival_time  
FROM flights  
WHERE act_arrival_time > sch_arrival_time;  
  
--8  
SELECT * FROM airline  
WHERE airline_country IN ('France', 'Portugal', 'Poland')  
AND created_at BETWEEN '2023-11-01' AND '2024-03-31';
```
- Data Output:** A table showing the results of the query. The columns are: airline_id, airline_code, airline_name, airline_country, created_at, and updated_at. The table is empty.
- Messages:** A status bar at the bottom showing 'Total rows: 0' and 'Query complete 00:00:00.054'. A green message box indicates 'Successfully run. Total query runtime: 54 msec. 0 rows affected.'

9. Find the top 3 overweighted baggage records with more than 25 kg.

```
SELECT *  
FROM baggage  
WHERE weight_in_kg > 25  
ORDER BY weight_in_kg DESC  
LIMIT 3;
```

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, including tables like 'airline', 'baggage', 'boarding_pass', 'booking', 'booking_flight', and 'flights'. The 'baggage' table is selected, showing its columns: 'baggage_id' (integer), 'weight_in_kg' (numeric(4,2)), 'created_at' (timestamp without time zone), 'updated_at' (timestamp without time zone), and 'booking_id' (integer).

The main query editor displays the following SQL query:

```
28 SELECT first_name, last_name,  
29        TO_CHAR(date_of_birth, 'month DD, YYYY')  
30 FROM passengers;  
31  
32 --6  
33 SELECT flight_id  
34 FROM flights  
35 WHERE act_arrival_time > sch_arrival_time;  
36  
37 --7  
38 SELECT flight_id, sch_arrival_time, act_arrival_time  
39 FROM flights  
40 WHERE act_arrival_time > sch_arrival_time;  
41  
42 --8  
43 SELECT * FROM airline  
44 WHERE airline_country IN ('France', 'Portugal', 'Poland')  
45 AND created_at BETWEEN '2023-11-01' AND '2024-03-31';  
46  
47 --9  
48 SELECT * FROM baggage  
49 WHERE weight_in_kg > 25  
50 ORDER BY weight_in_kg DESC  
51 LIMIT 3;  
52  
53
```

The 'Data Output' pane shows the results of the query, displaying 3 rows of baggage records:

baggage_id [PK] integer	weight_in_kg numeric (4,2)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer
1	1006	2026-09-26 19:45:13.617205	2026-09-26 19:45:13.617205	1006
2	1059	2026-09-26 19:45:13.617205	2026-09-26 19:45:13.617205	1059
3	1017	2026-09-26 19:45:13.617205	2026-09-26 19:45:13.617205	1017

The status bar at the bottom indicates: 'Total rows: 3', 'Query complete 00:00:00.063', and a green message box stating 'Successfully run. Total query runtime: 63 msec. 3 rows affected.'

10. Find the youngest passenger's full name.

```
SELECT first_name, last_name
FROM passengers
ORDER BY date_of_birth DESC
LIMIT 1;
```

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane lists database objects, with 'Columns (12)' expanded under the 'passengers' table. The main editor displays a SQL query with line numbers 37 to 63. The query is as follows:

```
37 WHERE airline_code IN ('France', 'Poland')
38 AND created_at BETWEEN '2023-11-01' AND '2023-11-31';
39
40 --9
41 SELECT * FROM baggage
42 WHERE weight_in_kg > 25
43 ORDER BY weight_in_kg DESC
44 LIMIT 3;
45
46 --10
47 SELECT first_name, last_name
48 FROM passengers
49 ORDER BY date_of_birth DESC
50 LIMIT 1;
51
52 --11
53 SELECT booking_platform, MIN(ticket_price)
54 FROM booking
55 GROUP BY booking_platform;
56
57 --12
58 -- SELECT * FROM airline
59 -- WHERE airline_code LIKE '%[0-9]%'
60
61 SELECT *
62 FROM airline
63 WHERE airline_code = '100-011';
```

The 'Data Output' pane at the bottom shows the results of the query on line 47. It displays a table with two columns: 'first_name' and 'last_name', both of type 'character varying (50)'. The first row contains the values 'First1093' and 'Last1093'.

first_name	last_name
First1093	Last1093

A status bar at the bottom right indicates: 'Successfully run. Total query runtime: 57 msec. 1 rows affected.' The bottom of the IDE shows 'Total rows: 1' and 'Query complete 00:00:00.057'.

11. Find the cheapest booking price on each booking platform.

```
SELECT booking_platform, MIN(ticket_price)
FROM booking
GROUP BY booking_platform;
```

The screenshot shows a PostgreSQL IDE interface. On the left is the Object Explorer with a tree view of database objects. The 'booking' table is selected, showing its columns: booking_id, flight_id, passenger_id, booking_platform, created_at, updated_at, status, and ticket_price. The main query editor displays a SQL query that finds the minimum ticket price for each booking platform. The query is as follows:

```
--7
SELECT flight_id, sch_arrival_time, act_arrival_time
FROM flights
WHERE act_arrival_time > sch_arrival_time;

--8
SELECT * FROM airline
WHERE airline_country IN ('France', 'Portugal', 'Poland')
AND created_at BETWEEN '2023-11-01' AND '2024-03-31';

--9
SELECT * FROM baggage
WHERE weight_in_kg > 25
ORDER BY weight_in_kg DESC
LIMIT 3;

--10
SELECT first_name, last_name
FROM passengers
ORDER BY date_of_birth DESC
LIMIT 1;

--11
SELECT booking_platform, MIN(ticket_price)
FROM booking
GROUP BY booking_platform;
```

Below the query editor, the 'Data Output' tab shows the results of the query. The results are displayed in a table with two columns: booking_platform and min. The table contains two rows:

booking_platform	min
Offline	8642.52
Online	9096.41

At the bottom of the IDE, a status bar indicates 'Total rows: 2' and 'Query complete 00:00:00.085'. A green message box at the bottom right states: 'Successfully run. Total query runtime: 85 msec. 2 rows affected.'

12. Return airlines whose airline_code contains a digit.

```
SELECT *  
FROM airline  
WHERE airline_code ~ '[0-9]';
```

The screenshot shows a PostgreSQL IDE interface. On the left is the Object Explorer showing a database schema with tables like airport, baggage, booking, flights, passengers, and security_check. The main pane displays a SQL query editor with the following content:

```
--9  
SELECT * FROM baggage  
WHERE weight_in_kg > 25  
ORDER BY weight_in_kg DESC  
LIMIT 3;  
  
--10  
SELECT first_name, last_name  
FROM passengers  
ORDER BY date_of_birth DESC  
LIMIT 1;  
  
--11  
SELECT booking_platform, MIN(ticket_price)  
FROM booking  
GROUP BY booking_platform;  
  
--12  
-- SELECT * FROM airline  
-- WHERE airline_code LIKE '%[0-9]%';  
  
SELECT *  
FROM airline  
WHERE airline_code ~ '[0-9]';  
  
--13
```

Below the query editor is the Data Output pane, which shows the results of the query. It displays a table with 6 columns: airline_id, airline_code, airline_name, airline_country, created_at, and updated_at. The results show 6 rows of data, all with airline_codes containing digits (R1001 through R1006). A status bar at the bottom indicates "Total rows: 199" and "Query complete 00:00:00.065". A green notification box at the bottom right states "Successfully run. Total query runtime: 65 msec. 199 rows affected."

airline_id	airline_code	airline_name	airline_country	created_at	updated_at
1	1001	R1001	Random Airline 1001	Country 20	2026-09-26 19:43:41.070387
2	1002	R1002	Random Airline 1002	Country 6	2026-09-26 19:43:41.070387
3	1003	R1003	Random Airline 1003	Country 8	2026-09-26 19:43:41.070387
4	1004	R1004	Random Airline 1004	Country 12	2026-09-26 19:43:41.070387
5	1005	R1005	Random Airline 1005	Country 12	2026-09-26 19:43:41.070387
6	1006	R1006	Random Airline 1006	Country 15	2026-09-26 19:43:41.070387

13. List the top 5 most recently created airlines.

```
SELECT airline_name, created_at
FROM airline
ORDER BY created_at DESC
LIMIT 5;
```

The screenshot shows a PostgreSQL IDE interface. On the left is the Object Explorer with a tree view of the database schema. The 'airline' table is selected under 'Tables (11)'. The 'Columns (6)' for the 'airline' table are listed: 'airline_id' (integer), 'airline_code' (character varying(30)), 'airline_name' (character varying(50)), 'airline_country' (character varying(50)), 'created_at' (timestamp without time zone), and 'updated_at' (timestamp without time zone). The main query editor displays a SQL query with line numbers 51 to 76. The query is:
51 LIMIT 3;
52
53 --10
54 SELECT first_name, last_name
55 FROM passengers
56 ORDER BY date_of_birth DESC
57 LIMIT 1;
58
59 --11
60 SELECT booking_platform, MIN(ticket_price)
61 FROM booking
62 GROUP BY booking_platform;
63
64 --12
65 -- SELECT * FROM airline
66 -- WHERE airline_code LIKE '%[0-9]%;'
67
68 SELECT *
69 FROM airline
70 WHERE airline_code ~ '[0-9]';
71
72 --13
73 SELECT airline_name, created_at
74 FROM airline
75 ORDER BY created_at desc
76 LIMIT 5;
The 'Data Output' tab is active, showing a table with 5 rows. The columns are 'airline_name' (character varying (50)) and 'created_at' (timestamp without time zone). The rows are: 1 Sky Air (2026-09-30 12:13:33.564644), 2 Sky Airlines (2026-09-30 11:11:00.008316), 3 Random Airline 1003 (2026-09-26 19:43:41.070387), 4 Random Airline 1002 (2026-09-26 19:43:41.070387), and 5 Random Airline 1001 (2026-09-26 19:43:41.070387). A status bar at the bottom indicates 'Total rows: 5', 'Query complete 00:00:00.059', and a green message box says 'Successfully run. Total query runtime: 59 msec. 5 rows affected.'

Object Explorer

- Collations
- Domains
- FTS Configurations
- FTS Dictionaries
- FTS Parsers
- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (11)
 - airline
- Columns (6)
 - airline_id integer
 - airline_code character varying(30)
 - airline_name character varying(50)
 - airline_country character varying(50)
 - created_at timestamp without time zone
 - updated_at timestamp without time zone
- Constraints
- Indexes
- RLS Policies
- Rules
- Triggers
- airport
- baggage
- baggage_check
- boarding_pass
- booking
- booking_flight
- flights
- passengers
- security_check
- students

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × postgres/postgres@PostgreSQL 18* ×

postgres/postgres@PostgreSQL 18

Query Query History Scratch Pad ×

```
51 LIMIT 3;  
52  
53 --10  
54 SELECT first_name, last_name  
55 FROM passengers  
56 ORDER BY date_of_birth DESC  
57 LIMIT 1;  
58  
59 --11  
60 SELECT booking_platform, MIN(ticket_price)  
61 FROM booking  
62 GROUP BY booking_platform;  
63  
64 --12  
65 -- SELECT * FROM airline  
66 -- WHERE airline_code LIKE '%[0-9]%;'  
67  
68 SELECT *  
69 FROM airline  
70 WHERE airline_code ~ '[0-9]';  
71  
72 --13  
73 SELECT airline_name, created_at  
74 FROM airline  
75 ORDER BY created_at desc  
76 LIMIT 5;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	airline_name character varying (50)	created_at timestamp without time zone
1	Sky Air	2026-09-30 12:13:33.564644
2	Sky Airlines	2026-09-30 11:11:00.008316
3	Random Airline 1003	2026-09-26 19:43:41.070387
4	Random Airline 1002	2026-09-26 19:43:41.070387
5	Random Airline 1001	2026-09-26 19:43:41.070387

Total rows: 5 Query complete 00:00:00.059

✓ Successfully run. Total query runtime: 59 msec. 5 rows affected. ✕

LF Ln 73, Col 1

14. Return all rows where `booking_id` is between 200 and 300 inclusive and `check_result` is not 'Checked'.

```
SELECT *  
FROM baggage_check  
WHERE booking_id BETWEEN 200 AND 300  
      AND check_result <> 'Checked';
```

The screenshot shows a PostgreSQL IDE interface. On the left is the Object Explorer showing a database schema with tables like `airline`, `airport`, `baggage`, `baggage_check`, `boarding_pass`, and `booking`. The `baggage_check` table is selected, showing its columns: `baggage_check_id` (integer), `check_result` (character varying), `created_at` (timestamp without time zone), `updated_at` (timestamp without time zone), `booking_id` (integer), and `passenger_id` (integer). The main query editor displays a SQL query with line numbers 56 to 81. The query is a SELECT statement that filters rows from the `baggage_check` table based on `booking_id` being between 200 and 300 and `check_result` not being 'Checked'. Below the query editor is a Data Output table with columns: `baggage_check_id` [PK] integer, `check_result` character varying (50), `created_at` timestamp without time zone, `updated_at` timestamp without time zone, `booking_id` integer, and `passenger_id` integer. The status bar at the bottom indicates 'Total rows: 0', 'Query complete 00:00:00.082', and a green message box stating 'Successfully run. Total query runtime: 82 msec. 0 rows affected.'

```
56 ORDER BY date_of_birth DESC  
57 LIMIT 1;  
58  
59 --11  
60 SELECT booking_platform, MIN(ticket_price)  
61 FROM booking  
62 GROUP BY booking_platform;  
63  
64 --12  
65 -- SELECT * FROM airline  
66 -- WHERE airline_code LIKE '%[0-9]%;'  
67  
68 SELECT *  
69 FROM airline  
70 WHERE airline_code ~ '[0-9]';  
71  
72 --13  
73 SELECT airline_name, created_at  
74 FROM airline  
75 ORDER BY created_at desc  
76 LIMIT 5;  
77  
78 --14  
79 SELECT * FROM baggage_check  
80 WHERE booking_id BETWEEN 200 AND 300  
81 AND check_result <> 'Checked';
```

baggage_check_id	check_result	created_at	updated_at	booking_id	passenger_id
------------------	--------------	------------	------------	------------	--------------

✓ Successfully run. Total query runtime: 82 msec. 0 rows affected. ✕

Total rows: 0 Query complete 00:00:00.082 LF Ln 79, Col 1

15. Find baggage checks where updated_at is in the same month as created_at, but updated_at is earlier than created_at.

```
SELECT *  
FROM baggage_check  
WHERE DATE_TRUNC('month', updated_at) = DATE_TRUNC('month', created_at)  
      AND updated_at < created_at;
```

The screenshot shows a PostgreSQL IDE interface. On the left is the Object Explorer showing the database schema. The main window is the Query Editor, which contains the following SQL query:

```
62 GROUP BY booking_platform;  
63  
64 --12  
65 -- SELECT * FROM airline  
66 -- WHERE airline_code LIKE '%[0-9]%;'  
67  
68 SELECT *  
69 FROM airline  
70 WHERE airline_code ~ '[0-9]';  
71  
72 --13  
73 SELECT airline_name, created_at  
74 FROM airline  
75 ORDER BY created_at desc  
76 LIMIT 5;  
77  
78 --14  
79 SELECT * FROM baggage_check  
80 WHERE booking_id BETWEEN 200 AND 300  
81       AND check_result <> 'Checked';  
82  
83 --15  
84 SELECT *  
85 FROM baggage_check  
86 WHERE DATE_TRUNC('month', updated_at) = DATE_TRUNC('month', created_at)  
87       AND updated_at < created_at;
```

Below the query editor is the Data Output section, which shows a table with the following columns:

baggage_check_id	check_result	created_at	updated_at	booking_id	passenger_id
[PK] integer	character varying (50)	timestamp without time zone	timestamp without time zone	integer	integer

At the bottom of the interface, a status bar indicates: "Total rows: 0 Query complete 00:00:00.052". A green notification box at the bottom right states: "✓ Successfully run. Total query runtime: 52 msec. 0 rows affected. X".